

Umbrella Notes — Module 3

DBMS & SQL | shared core for Coal India MT (Systems), IBPS SO IT Officer (SPL-XVI), DSSSB TGT CS (DOE)

How to read this module

This is **full teaching notes** — learn directly from it. Depth is **balanced across all three exams** (middle depth by default: enough for IBPS SO, DSSSB TGT, and general PSU/Govt CS-IT-Systems exams).

■ **Green box** = how PSU/Govt exams actually test the topic (a reference lens, not a limit).

↑ **Amber box** = extra GATE-core depth needed *only* if targeting Coal India MT (or GATE-style PSUs). Skip for IBPS/DSSSB.

■ **Purple box** = a coverage warning (something kept light, or where an exam needs more/less).

Small sub-topics are folded under umbrella headings but never dropped. Topic-wise Q&A with explanations comes as a separate set after your walkthrough.

0 What this module covers

A **Database Management System (DBMS)** is software that lets you store, organise, retrieve and manage large amounts of related data efficiently and safely. It replaces the old 'files scattered across programs' approach with one controlled, shared, consistent store. The relational model dominates, so this module is built around it.

- **1. Fundamentals** — what a DBMS is, why it beats file systems, architecture, keys.
- **2. Data models & the relational model** — relations, attributes, tuples, ER modelling.
- **3. Normalization** — removing redundancy via 1NF–3NF/BCNF.
- **4. SQL** — DDL/DML/DCL/TCL, queries, joins, key functions.
- **5. Transactions & concurrency** — ACID, schedules, locking.
- **6. Indexing & file organization** — how data is stored and found fast.

■ Tested pattern & focus (PSU / Govt CS-IT-Systems)

DBMS is a **high-frequency, high-scoring** area for IBPS SO IT, DSSSB TGT and PSU CS papers — mostly direct concept MCQs plus **SQL output** questions.

Guaranteed clusters: **keys** (primary/candidate/foreign), **normal forms**, **ACID properties**, **types of joins**, **SQL commands classification** (DDL vs DML), and **normalization anomalies**.

1 Fundamentals & Architecture

1.1 DBMS vs file system

Traditional file processing stored data in separate files per application, causing problems a DBMS fixes:

Problem in file systems	How DBMS solves it
Data redundancy & inconsistency	Central store; controlled/'normalized' data
Difficult data access	Query language (SQL)
No concurrent access control	Transaction & locking mechanisms
Poor security & integrity	Access control + integrity constraints
No backup/recovery	Built-in recovery from failures

1.2 Three-schema (ANSI-SPARC) architecture

A DBMS separates data into three levels to give **data abstraction** and **independence**:

- **External / View level** — what each user sees (custom views).
- **Conceptual / Logical level** — the whole database structure (tables, relationships).
- **Internal / Physical level** — how data is actually stored on disk.

Data independence is the payoff: **logical independence** = change the conceptual schema without touching views; **physical independence** = change storage without touching the logical schema.

1.3 Keys — learn these cold

Key	Meaning
Super key	Any set of attributes that uniquely identifies a tuple
Candidate key	A minimal super key (no redundant attribute)
Primary key	The chosen candidate key; unique + NOT NULL
Alternate key	Candidate keys not chosen as primary
Foreign key	An attribute referencing the primary key of another table (enforces referential integrity)
Composite key	A key made of two or more attributes

■ Tested pattern & focus (PSU / Govt CS-IT-Systems)

Key definitions are a near-guaranteed MCQ. The most-asked distinctions: candidate vs primary, super vs candidate (minimality), and what a **foreign key** enforces (referential integrity).

Remember: primary key = **unique AND not null**; a table can have many candidate keys but one primary key.

2 Data Models & the Relational Model

2.1 Terminology (relational)

Term	Plain meaning	Also called
Relation	A table	Table
Tuple	A row / record	Row
Attribute	A column / field	Column
Degree	Number of attributes (columns)	—
Cardinality	Number of tuples (rows)	—
Domain	Set of allowed values for an attribute	—

2.2 ER model (design stage)

The **Entity-Relationship (ER) model** is a diagram-based way to design a database before implementing it. Its building blocks: **entity** (a real-world object → rectangle), **attribute** (a property → ellipse), and **relationship** (an association → diamond). Relationships have a **cardinality**: **1:1**, **1:N**, **M:N**.

- **Key attribute** — underlined; uniquely identifies the entity.
- **Composite attribute** — divisible (e.g. Name → First+Last). **Multivalued** — double ellipse (e.g. phone numbers). **Derived** — dashed ellipse (e.g. age from DOB).
- **Weak entity** — can't be identified without a related (owner) entity; drawn with double rectangle.

↑ Go deeper — GATE-core (Coal India MT)

GATE-side: convert ER → relational schema systematically, handle **generalization/specialization** and aggregation, and work with **relational algebra** (selection σ , projection π , joins $\bowtie$, set ops) and **tuple relational calculus**. IBPS/DSSSB rarely go past ER symbols + cardinality.

3 Normalization

Normalization is the process of organising attributes into tables to **minimise redundancy** and avoid update **anomalies**. It's driven by **functional dependencies (FD)**: $X \rightarrow Y$ means 'X determines Y' (each X value fixes one Y value).

3.1 The anomalies normalization prevents

- **Insertion anomaly** — can't add data without other, unrelated data.
- **Update anomaly** — changing one fact requires many row edits (risking inconsistency).
- **Deletion anomaly** — deleting one fact accidentally loses another.

3.2 The normal forms

Form	Requirement (each builds on the previous)
1NF	Atomic (indivisible) values only; no repeating groups or multivalued attributes
2NF	1NF + no partial dependency (no non-key attribute depends on part of a composite key)
3NF	2NF + no transitive dependency (non-key attribute must not depend on another non-key attribute)
BCNF	Stricter 3NF: for every FD $X \rightarrow Y$, X must be a super key

■ Tested pattern & focus (PSU / Govt CS-IT-Systems)

"Which normal form removes X anomaly / dependency?" is the classic MCQ. Memorise: 2NF kills **partial** dependency, 3NF kills **transitive** dependency, BCNF requires the determinant to be a **super key**.

Mnemonic for the progression: *1NF atomic → 2NF no partial → 3NF no transitive → BCNF determinant is super key.*

↑ Go deeper — GATE-core (Coal India MT)

GATE-side: compute **attribute closure** (X^+), find **candidate keys** from an FD set, test a decomposition for **lossless join** and **dependency preservation**, and handle **4NF** (multivalued dependency). These are heavy for Coal India but out of scope for IBPS/DSSSB.

4 SQL — Structured Query Language

4.1 Command categories

Category	Full form	Commands	Purpose
DDL	Data Definition	CREATE, ALTER, DROP, TRUNCATE	Define/modify structure
DML	Data Manipulation	SELECT, INSERT, UPDATE, DELETE	Work with the data
DCL	Data Control	GRANT, REVOKE	Permissions
TCL	Transaction Control	COMMIT, ROLLBACK, SAVEPOINT	Manage transactions

■ Two traps exams love

DELETE vs TRUNCATE vs DROP: DELETE (DML) removes rows and can be rolled back; TRUNCATE (DDL) removes all rows fast, can't easily roll back; DROP (DDL) removes the whole table structure.

WHERE vs HAVING: WHERE filters rows *before* grouping; HAVING filters groups *after* GROUP BY.

4.2 Query anatomy & order

A SELECT is written as SELECT ... FROM ... WHERE ... GROUP BY ... HAVING ... ORDER BY, but is **logically executed** roughly: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY. Knowing this order explains why you can't use a column alias in WHERE but can in ORDER BY.

4.3 Joins

Join	Returns
INNER JOIN	Only rows matching in both tables
LEFT (OUTER) JOIN	All left-table rows + matches from right (NULLs if none)
RIGHT (OUTER) JOIN	All right-table rows + matches from left
FULL (OUTER) JOIN	All rows from both, matched where possible
CROSS JOIN	Cartesian product (every row × every row)
SELF JOIN	A table joined to itself

4.4 Functions (DSSSB DOE lists these explicitly)

- **Aggregate:** COUNT, SUM, AVG, MIN, MAX (often with GROUP BY).
- **String:** UPPER/UCASE, LOWER/LCASE, LENGTH, SUBSTR/MID, LEFT, RIGHT, TRIM, INSTR.
- **Numeric:** ROUND, POWER, MOD. **Date:** NOW, DATE, MONTH, YEAR, DAY, DAYNAME.

■ Tested pattern & focus (PSU / Govt CS-IT-Systems)

Two SQL question styles dominate: **classify the command** (is CREATE DDL or DML?) and **predict the output** of a small query (especially joins, GROUP BY/HAVING, and aggregate functions).

DSSSB (DOE/NDMC) explicitly tests **MySQL functions** (UCASE, MID, ROUND, MONTHNAME, etc.) — know what each returns.

5 Transactions & Concurrency Control

A **transaction** is a logical unit of work — a sequence of operations that must execute as one all-or-nothing step (the classic example: transferring money = debit + credit, both or neither).

5.1 ACID properties

Property	Meaning
Atomicity	All operations happen, or none do (no partial transaction)
Consistency	A transaction takes the DB from one valid state to another
Isolation	Concurrent transactions don't interfere; result = as if serial
Durability	Once committed, changes survive crashes/power loss

■ Tested pattern & focus (PSU / Govt CS-IT-Systems)

ACID is the single most-asked transaction MCQ — memorise all four names and one-line meanings. 'Which property guarantees committed data survives a crash?' → Durability. 'All-or-nothing?' → Atomicity.

5.2 Schedules & concurrency problems

Running transactions concurrently improves throughput but risks anomalies: **dirty read** (reading uncommitted data), **lost update**, and **unrepeatable read**. A **serializable** schedule is one whose effect equals some serial order — the correctness goal.

5.3 Concurrency control

- **Locking** — shared (read) & exclusive (write) locks. **Two-Phase Locking (2PL)** (a growing phase then a shrinking phase) guarantees serializability.
- **Timestamp ordering** — order transactions by timestamps.
- Deadlocks can arise between transactions (same idea as OS deadlocks) — handled by detection or prevention.

↑ Go deeper — GATE-core (Coal India MT)

GATE-side: test a schedule for **conflict-serializability** via a precedence graph, recoverable/cascadeless schedules, strict vs rigorous 2PL, and log-based recovery (undo/redo, checkpoints). IBPS/DSSSB stop at ACID + basic locking.

6 Indexing & File Organization

An **index** is an auxiliary structure that speeds up data retrieval — like a book's index, it lets the DBMS find rows without scanning the whole table (trading extra storage + slower writes for faster reads).

- **Primary index** — on the ordering key of a sorted file. **Secondary index** — on a non-ordering attribute.
- **Dense index** — one entry per record; **sparse index** — one entry per block.
- **B-tree / B+ tree** — balanced multi-level index structures used by real databases; B+ tree keeps all data in leaf nodes linked for fast range scans.

■ Tested pattern & focus (PSU / Govt CS-IT-Systems)

Concept MCQs: purpose of an index (faster search, slower insert/update), **primary vs secondary** and **dense vs sparse**, and recognising that databases use **B+ trees** for indexing.

■ Coverage note / warning

File-organization internals (heap/sequential/hashing block math) are kept light here — low frequency for IBPS/DSSSB. For Coal India, pair this with the **B/B+ tree operations** in the amber box logic and the storage hierarchy in Module 5.

Rapid recap — highest-yield facts

Area	Nail these
Fundamentals	DBMS beats file systems (redundancy, access, concurrency, integrity); 3-schema (external/conceptual/internal); logical vs physical independence
Keys	Super $\supseteq$ candidate(minimal) $\rightarrow$ primary(unique+not null); foreign key = referential integrity
Relational	Relation=table, tuple=row, attribute=column, degree=#cols, cardinality=#rows; ER: entity/attribute/relationship, 1:1/1:N/M:N
Normalization	1NF atomic $\rightarrow$ 2NF no partial $\rightarrow$ 3NF no transitive $\rightarrow$ BCNF determinant=super key; fixes insert/update/delete anomalies
SQL	DDL(CREATE/ALTER/DROP/TRUNCATE), DML(SELECT/INSERT/UPDATE/DELETE), DCL(GRANT/REVOKE), TCL(COMMIT/ROLLBACK); WHERE before GROUP BY, HAVING after; join types
Transactions	ACID = Atomicity/Consistency/Isolation/Durability; serializability; 2PL; dirty read
Indexing	Index = faster read/slower write; primary vs secondary, dense vs sparse; B+ tree

✓ Before your walkthrough

That's the shared-core DBMS & SQL picture at balanced depth. Flag any page/section that felt shaky and I'll go deeper; then the **DBMS topic-wise Q&A set** (real PSU/Govt questions + explanations, including SQL output problems) follows for live practice.

Coal India target? The amber $\uparrow$ **GATE-core** boxes — relational algebra, attribute closure & candidate keys, lossless-join decomposition, conflict-serializability — are the extra depth DBMS needs.